

TensorForge 2.0

Phase 2: MVP Engineering Report

Automated Multilingual Customer Support Ticket Classification and Routing System

Item	Details
Prepared by	CTRL+C
Repository	https://github.com/TIHOR1140/Tensorforge-2.0.git
API_KEY	tf2_82e3d8feb90e34ba176854047b4c2755e1006bce47b52879
Demo video link	https://youtu.be/rXegHz7Trm0
Date	10 October 2026
Use case	RideEat multilingual support ticket classification and routing (English, Sinhala, Tamil, Singlish, Tanglish)

1. Executive Summary

TensorForge 2.0 Phase 2 delivers an offline, non-LLM ticket classification and routing engine for RideEat, a high-throughput multi-service platform serving customers in English, Sinhala, Tamil, Singlish and Tanglish. The solution has four parts:

- Offline non-LLM ML pipeline.** Multilingual n-gram TF-IDF features feed three calibrated logistic-regression heads, followed by a deterministic business rule engine.
- Contract-compliant FastAPI gateway.** Follows the OpenAPI 3.0.3 spec and JSON Schema 2020-12, with constant-time API key checks, an ordered error hierarchy (401, 415, 413, 400, 422), X-Request-ID tracing and async batch jobs of up to 5,000 tickets.
- React operations dashboard.** Live single-ticket triage, batch CSV upload, confidence monitoring and manual escalation.
- Zero-dependency Docker image.** Starts in under 15 seconds with no runtime internet: `docker run -p 8000:8000 -e API_KEY=<key> <image>`.

Headline Results (800-sample validation set)

Metric	Baseline	Trained Engine	Improvement
Primary Category Macro F1	0.4120	0.6568	+59.4%
Secondary Category Macro F1	0.2840	0.7069	+148.9%
Urgency Binary F1	0.5210	0.8679	+66.6%
Urgency Recall	0.5875	0.8625	+46.8%
Urgency Overall Accuracy	0.8850	0.9738	+10.0%
Schema Validation Compliance	100%	100%	Perfect compliance

2. Core Concept & System Architecture

The core idea is to decompose ticket understanding into three specialised statistical heads that share one multilingual feature space, then pass their outputs through a deterministic rule engine so every response is valid, routable and consistent. The system is split into four layers:

Layer	Responsibility
Presentation Layer	React/Vite dashboard served at root; single triage, CSV bulk testing, charts.
Ingestion & Gateway Layer	FastAPI on port 8000; ordered validation pipeline (Auth, Content-Type, Size, JSON syntax, Schema).
Execution & Job Manager Layer	Route dispatcher for GET /health, POST /predict, POST /predict/batch (1-100) and POST /batch/jobs (2,000-5,000, async); thread-pool workers.
Statistical Inference & Consistency Engine	Preprocessor, TF-IDF vectorizer, three model heads and the business rule engine (rules.py).

3. Key Design Decisions

3.1 Strict error-handling precedence

Security must run before validation. Middleware intercepts raw requests before JSON decoding and enforces Auth (401) → MediaType (415) → PayloadSize (413) → JSONSyntax (400) → Schema (422). This stops malformed payloads from bypassing authentication and closes a denial-of-service path.

3.2 Asynchronous, non-blocking batch processing

Holdout files contain 2,000 to 5,000 tickets, and synchronous HTTP over that volume risks timeouts. POST /batch/jobs therefore returns:

- an immediate HTTP 202 with a unique job_id;
- Idempotency-Key deduplication, so identical submissions never run twice;
- background thread-pool execution with atomic in-memory status (queued → processing → succeeded or failed);
- chunked retrieval via GET /batch/jobs/{job_id}/results?offset=X&limit=Y, keeping memory bounded.

3.3 Guaranteed offline containerisation

No network calls happen at runtime. Vocabulary, model coefficients, calibrators and the frontend bundle are baked into the image layers. /health returns HTTP 200 in under 15 s, well inside the 120 s limit.

3.4 Why a statistical model instead of an LLM

- **Compliance and offline operation:** the guidelines scrutinise LLM wrappers and cloud endpoints break the offline requirement.
- **Throughput:** local LLMs need heavy GPUs, cold-start in over 120 s and reach under 20 req/s, against over 450 tickets/s here.
- **Determinism:** routing needs guaranteed enums and team mappings, which generative output cannot promise.

4. Machine Learning Methodology

4.1 Dataset

train.csv has 4,000 rows and validation.csv has 800. Language mix: English 35%, Singlish 25%, Sinhala 20%, Tamil 15%, Tanglish 2.5%, Mixed 2.5%. Challenges include inconsistent transliteration, colloquial phrasing (e.g. “driver awilla na salli kapala”), embedded newlines, typos, code-switching and adversarial spam. Intents also overlap: a late order with a refund demand is primarily delivery_delay with payment_refund as secondary.

4.2 Feature engineering

- **Channel synthesis:** input is built as [CHANNEL: c] [SUBJECT: s] text, so the model learns that emails are formal and chats or call transcripts are terse and urgent.
- **Unicode normalisation:** handles combining marks and ZWJ/ZWNJ in Sinhala and Tamil, lowercases, strips control characters and keeps distress-signalling punctuation.
- **Sublinear n-gram TF-IDF:** (1,2)-grams, 25,000 features, $TF-IDF = (1 + \log tf) \times \log((1 + n) / (1 + df))$. It covers Latin transliterations (salli, awilla) and native tokens (පුරකිකු, மிகவும் அவசரம்) alike.

4.3 Multi-head architecture

	Head 1: Primary Category	Head 2: Secondary Category	Head 3: Urgency Detector
Model	Logistic Regression, C = 2.5	Logistic Regression, C = 1.5	Logistic Regression, C = 2.0
Calibration	CalibratedClassifierCV, 3-fold	None (direct multi-class)	CalibratedClassifierCV, 3-fold
Output	11 classes	11 classes + synthetic 'none'	Binary, recall-biased
Weights	Balanced	Balanced	Balanced, biased to recall
Purpose	Main intent and team routing	Multi-intent detection	Safety escalation, fewer missed emergencies

5. Business Consistency & Safety Rule Engine

A deterministic engine (backend/models/rules.py) post-processes model output. Every primary category maps to exactly one team (below), and three invariants hold: (1) a secondary category equal to the primary is reset to null; (2) spam_irrelevant forces is_urgent = False and secondary = null; (3) prompt-injection phrases such as “Ignore all previous instructions” cannot escalate priority, because they are handled by feature scaling and lexical grounding.

Category	Support Team	Focus
payment_refund	Payments & Refunds	Financial auditing, gateway disputes
ride_trip_issue	Ride Operations	Route deviations, fare adjustments
lost_item	Lost & Found	Driver contact, item retrieval
order_missing_wrong	Food Operations	Restaurant verification, re-dispatch
delivery_delay	Delivery Operations	Rider dispatch, re-routing
food_quality	Restaurant Quality	Food safety, hygiene audits
account_promo	Account Services	OTP, voucher and login issues
safety_conduct	Trust & Safety	Incident review, suspension
app_technical	Tech Support	Crash logs, payment error codes
general_inquiry	Front-line Support	Onboarding, FAQs, partner payouts
spam_irrelevant	Auto-close / Spam Filter	Automated termination, bot filtering

6. Empirical Evaluation

The pipeline was evaluated on the official 800-sample validation set, which was not used in training.

6.1 Validation performance summary

Metric	Baseline	Trained Engine	Relative Improvement
Primary Macro F1	0.4120	0.6568	+59.4%
Primary Accuracy	0.4350	0.6275	+44.3%
Primary Weighted F1	0.4285	0.6340	+47.9%
Secondary Macro F1	0.2840	0.7069	+148.9%
Urgency Binary F1	0.5210	0.8679	+66.6%
Urgency Overall Accuracy	0.8850	0.9738	+10.0%
Urgency Detection Recall	0.5875	0.8625	+46.8%
Schema Compliance	100%	100%	Perfect

6.2 Per-class performance (primary category)

Class	Precision	Recall	F1-Score	Support
account_promo	0.8305	0.6806	0.7481	72
app_technical	0.8649	0.5000	0.6337	64
delivery_delay	0.6320	0.6583	0.6449	120
food_quality	0.8333	0.5357	0.6522	56
general_inquiry	0.5424	0.8000	0.6465	40
lost_item	0.3846	0.7500	0.5085	40
order_missing_wrong	0.3431	0.3646	0.3535	96
payment_refund	0.5570	0.5764	0.5666	144
ride_trip_issue	0.8713	0.7857	0.8263	112
safety_conduct	0.6667	0.6250	0.6452	32
spam_irrelevant	1.0000	1.0000	1.0000	24
Accuracy			0.6275	800
Macro avg	0.6842	0.6615	0.6568	800
Weighted avg	0.6635	0.6275	0.6340	800

7. Linguistic Cohort Analysis & Key Findings

Language	Description	Samples	Primary Macro F1	Urgency F1
singlish	Sinhala transliterated in Latin script	200	0.9062	0.9714
en	Standard English	280	0.6133	0.6667
ta	Tamil Unicode	120	0.6112	1.0000
tanglish	Tamil transliterated in Latin script	20	0.5610	0.6667
si	Sinhala Unicode	160	0.4501	1.0000
mixed	Native Unicode mixed with English	20	0.4225	1.0000

- **Transliteration strength:** Singlish reaches 0.9062 Macro F1; n-grams capture phonetic variants such as “mata salli epa” and “rider awe na”.
- **Native-script safety:** Urgency F1 is 1.0000 on pure Sinhala and Tamil, so no emergency was missed for those users. Tanglish and mixed (n = 20 each) are small cohorts, so their scores are indicative only.
- **Weakest area:** Sinhala Unicode (0.4501) and mixed-script (0.4225) primary F1 show where character n-grams run out of semantic depth.
- **Intent overlap:** order_missing_wrong (0.3535) and payment_refund (0.5666) are confused because customers with missing items often demand refunds. The secondary head recovers this link (Secondary Macro F1 0.7069).

8. Operational Readiness

Checklist Item	Status	Evidence
JSON Schema 2020-12 validation	Passed	scripts/validate_schemas.py: all 4 schemas
API key auth on prediction/job endpoints	Passed	Constant-time check; test_endpoints.py: all 9 suites
Public /health endpoint	Passed	HTTP 200, status ok, model_loaded true
Single-command offline Docker run	Passed	Starts in under 15 s
Async batch for holdout jobs	Passed	Up to 5,000 tickets, polling and pagination
Live frontend demo	Passed	React/Vite dashboard at /
Training and validation documented	Passed	ml/training/train.py, no LLM wrapping
Video walkthrough	Ready	15-minute presentation script

9. Conclusion & Roadmap

By favouring disciplined multilingual statistical modelling and deterministic rule enforcement over LLM wrappers, TensorForge 2.0 delivers a fast, fully offline, schema-compliant and auditable routing system. Planned next steps:

- **Edge transformer distillation:** a quantised multilingual MiniLM or distilled XLM-RoBERTa (under 100 MB) on CPU to improve Sinhala Unicode nuance.
- **Active learning loop:** route needs_human_review tickets (confidence < 0.65) into continuous fine-tuning from agent corrections.
- **Voice normalisation:** real-time phonetic alignment to reduce spelling variance in call transcripts.